

```

        IngestionClient.Tenant,
        iapId,
        submissionId),
        requestContent: null).Wait();

    // Periodically check the status until it changes from
    "CommitsStarted" to either
    // successful status or a failure.
    Console.WriteLine("Waiting for the submission commit processing
    to complete. This may take a couple of minutes.");
    string submissionStatus = null;
    do
    {
        Task.Delay(TimeSpan.FromSeconds(5)).Wait();
        dynamic statusResource = client.Invoke<dynamic>(
            HttpMethod.Get,
            relativeUrl: string.Format(
                CultureInfo.InvariantCulture,
                IngestionClient.InAppSubmissionStatusUrlTemplate,
                IngestionClient.Version,
                IngestionClient.Tenant,
                iapId,
                submissionId),
            requestContent: null).Result;

        submissionStatus = statusResource.status.Value as string;
        Console.WriteLine("Current status: " + submissionStatus);
    }
    while ("CommitStarted".Equals(submissionStatus));

    if ("CommitFailed".Equals(submissionStatus))
    {
        Console.WriteLine("Submission has failed. Please check the
        Errors collection of the submissionResource response.");
        return;
    }
    else
    {
        Console.WriteLine("Submission commit success!");
    }
}
}
}

```

Update an add-on submission

The following example implements a class that uses several methods in the Microsoft Store submission API to update an existing add-on submission. The

`RunInAppProductSubmissionUpdateSample` method in the class creates a new submission

as a clone of the last published submission, and then it updates and commits the cloned

submission to Partner Center. Specifically, the `RunInAppProductSubmissionUpdateSample` method performs these tasks:

1. To begin, the method [gets data for the specified add-on](#).
2. Next, it [deletes the pending submission for the add-on](#), if one exists.
3. It then [creates a new submission for the add-on](#) (the new submission is a copy of the last published submission).
4. Next, it [updates](#) and then [commits](#) the new submission to Partner Center.
5. Finally, it periodically [checks the status of the new submission](#) until the submission is successfully committed.

C#

```
namespace DeveloperApiCSharpSample
{
    using System;
    using System.Globalization;
    using System.Net.Http;
    using System.Threading.Tasks;

    /// <summary>
    /// Sample code for how to update add-on submissions
    /// </summary>
    public class InAppProductSubmissionUpdateSample
    {
        private ClientConfiguration ClientConfig;

        /// <summary>
        /// Constructor
        /// </summary>
        /// <param name="c">An instance of ClientConfiguration that contains
        all parameters populated</param>
        public InAppProductSubmissionUpdateSample(ClientConfiguration c)
        {
            this.ClientConfig = c;
        }

        public void RunInAppProductSubmissionUpdateSample()
        {
            // *****
            //         SETTINGS
            // *****
            var iapId = this.ClientConfig.InAppProductId;
            var clientId = this.ClientConfig.ClientId;
            var clientSecret = this.ClientConfig.ClientSecret;
            var serviceEndpoint = this.ClientConfig.ServiceUrl;
            var tokenEndpoint = this.ClientConfig.TokenEndpoint;

            // Get authorization token
            Console.WriteLine("Getting authorization token ");
            var accessToken =
```

```

IngestionClient.GetClientCredentialAccessToken(
    tokenEndpoint,
    clientId,
    clientSecret).Result;

Console.WriteLine("Getting the add-on");
var client = new IngestionClient(accessToken, serviceEndpoint);
dynamic iap = client.Invoke<dynamic>(
    HttpMethod.Get,
    relativeUrl: string.Format(
        CultureInfo.InvariantCulture,
        IngestionClient.GetInAppUrlTemplate,
        IngestionClient.Version,
        IngestionClient.Tenant,
        iapId),
    requestContent: null).Result;
Console.WriteLine(iap.ToString());

// Let's see if there is a pending submission. Warning! If it
was created through the API,
// it will be deleted so that we could create a new one in its
stead.
if (iap.pendingInAppProductSubmission != null)
{
    var submissionId =
iap.pendingInAppProductSubmission.id.Value as string;

    // Let's try deleting it. If it was NOT created via the API,
then you need to manually
    // delete it from the dashboard. This is a safety measure to
make sure that a human user and
    // an automated system don't make conflicting edits.
    Console.WriteLine("Deleting the pending submission");

    client.Invoke<dynamic>(
        HttpMethod.Delete,
        relativeUrl: string.Format(
            CultureInfo.InvariantCulture,
            IngestionClient.InAppSubmissionUrlTemplate,
            IngestionClient.Version,
            IngestionClient.Tenant,
            iapId,
            submissionId),
        requestContent: null).Wait();
}

// Create a new submission, which will be an exact copy of the
last published submission.
Console.WriteLine("Creating a new submission");
dynamic clonedSubmission = client.Invoke<dynamic>(
    HttpMethod.Post,
    relativeUrl: string.Format(
        CultureInfo.InvariantCulture,
        IngestionClient.InAppSubmissionUrl,
        IngestionClient.Version,

```

```

        IngestionClient.Tenant,
        iapId),
        requestContent: null).Result;
var clonedSubmissionId = clonedSubmission.id.Value as string;
Console.WriteLine(clonedSubmission.ToString());

// Update the add-on price and keep the rest unchanged.
clonedSubmission.pricing.priceId = "Tier2"; // $0.99

// Because we are not uploading any new images, we don't need to
upload the zip file.

// Update the submission.
Console.WriteLine("Updating the submission");
client.Invoke<dynamic>(
    HttpMethod.Put,
    relativeUrl: string.Format(
        CultureInfo.InvariantCulture,
        IngestionClient.InAppSubmissionUrlTemplate,
        IngestionClient.Version,
        IngestionClient.Tenant,
        iapId,
        clonedSubmissionId),
    requestContent: clonedSubmission).Wait();

// Tell the system that we are done updating the submission.
Console.WriteLine("Committing the submission");
client.Invoke<dynamic>(
    HttpMethod.Post,
    relativeUrl: string.Format(
        CultureInfo.InvariantCulture,
        IngestionClient.InAppProductCommitSubmissionUrlTemplate,
        IngestionClient.Version,
        IngestionClient.Tenant,
        iapId,
        clonedSubmissionId),
    requestContent: null).Wait();

// Periodically check the status until it changes from
"CommitsStarted" to either
// successful status or a failure.
Console.WriteLine("Waiting for the submission commit processing
to complete. This may take a couple of minutes.");
string submissionStatus = null;
do
{
    Task.Delay(TimeSpan.FromSeconds(5)).Wait();
    dynamic statusResource = client.Invoke<dynamic>(
        HttpMethod.Get,
        relativeUrl: string.Format(
            CultureInfo.InvariantCulture,
            IngestionClient.InAppSubmissionStatusUrlTemplate,
            IngestionClient.Version,
            IngestionClient.Tenant,
            iapId,

```

```

        clonedSubmissionId),
        requestContent: null).Result;

        submissionStatus = statusResource.status.Value as string;
        Console.WriteLine("Current status: " + submissionStatus);
    }
    while ("CommitStarted".Equals(submissionStatus));

    if ("CommitFailed".Equals(submissionStatus))
    {
        Console.WriteLine("Submission has failed. Please check the
Errors collection of the submissionResource response.");
        return;
    }
    else
    {
        Console.WriteLine("Submission commit success! Here is the
new price:");
        dynamic sub = client.Invoke<dynamic>(
            HttpMethod.Get,
            relativeUrl: string.Format(
                CultureInfo.InvariantCulture,
                IngestionClient.InAppSubmissionUrlTemplate,
                IngestionClient.Version,
                IngestionClient.Tenant,
                iapId,
                clonedSubmissionId),
            requestContent: null).Result;
        Console.WriteLine(sub.pricing.priceId.Value as string);
    }
}
}
}
}

```

Create a package flight submission

The following example implements a class that uses several methods in the Microsoft Store submission API to update a package flight submission. The `RunFlightSubmissionUpdateSample` method in the class creates a new submission as a clone of the last published submission, and then it updates and commits the cloned submission to Partner Center. Specifically, the `RunFlightSubmissionUpdateSample` method performs these tasks:

1. To begin, the method [gets data for the specified package flight](#).
2. Next, it [deletes the pending submission for the package flight](#), if one exists.
3. It then [creates a new submission for the package flight](#) (the new submission is a copy of the last published submission).

4. It uploads a new package for the submission to Azure Blob Storage.
5. Next, it [updates](#) and then [commits](#) the new submission to Partner Center.
6. Finally, it periodically [checks the status of the new submission](#) until the submission is successfully committed.

C#

```
namespace DeveloperApiCSharpSample
{
    using System;
    using System.Collections.Generic;
    using System.Diagnostics;
    using System.Globalization;
    using System.Net.Http;
    using System.Threading.Tasks;
    using Newtonsoft.Json.Linq;

    /// <summary>
    /// Demonstrates how to update a flight submission with a new package
    /// </summary>
    public class FlightSubmissionUpdateSample
    {
        private ClientConfiguration ClientConfig { get; set; }

        /// <summary>
        /// Constructor
        /// </summary>
        /// <param name="c">An instance of ClientConfiguration that contains
all parameters populated</param>
        [DebuggerStepThrough]
        public FlightSubmissionUpdateSample(ClientConfiguration c)
        {
            this.ClientConfig = c;
        }

        public void RunFlightSubmissionUpdateSample()
        {
            // *****
            //      SETTINGS
            // *****
            var appId = this.ClientConfig.ApplicationId;
            var flightId = this.ClientConfig.FlightId;
            var clientId = this.ClientConfig.ClientId;
            var clientSecret = this.ClientConfig.ClientSecret;
            var serviceEndpoint = this.ClientConfig.ServiceUrl;
            var tokenEndpoint = this.ClientConfig.TokenEndpoint;
            var scope = this.ClientConfig.Scope;

            // Get authorization token
            Console.WriteLine("Getting authorization token ");
            var accessToken =
IngestionClient.GetClientCredentialAccessToken(
                tokenEndpoint,
```

```

        clientId,
        clientSecret,
        scope).Result;

Console.WriteLine("Getting flight");
var client = new IngestionClient(accessToken, serviceEndpoint);

dynamic flight = client.Invoke<dynamic>(
    HttpMethod.Get,
    relativeUrl: string.Format(
        CultureInfo.InvariantCulture,
        IngestionClient.GetFlightUrlTemplate,
        IngestionClient.Version,
        IngestionClient.Tenant,
        appId,
        flightId),
    requestContent: null).Result;
Console.WriteLine(flight.ToString());

if (flight.pendingFlightSubmission != null)
{
    var submissionId = flight.pendingFlightSubmission.id.Value
as string;

    // Let's try deleting it. If it was NOT creating via the
API, then you need to
    // manually delete it from the dashboard. This is a safety
measure to make sure that a
    // human user and an automated system don't make conflicting
edits.

    Console.WriteLine("Deleting the pending submission");

    client.Invoke<dynamic>(
        HttpMethod.Delete,
        relativeUrl: string.Format(
            CultureInfo.InvariantCulture,
            IngestionClient.GetFlightSubmissionUrlTemplate,
            IngestionClient.Version,
            IngestionClient.Tenant,
            appId,
            flightId,
            submissionId),
        requestContent: null).Wait();
}

// Create a new submission, which will be an exact copy of the
last published submission.
Console.WriteLine("Creating a new submission");
dynamic flightSubmission = client.Invoke<dynamic>(
    HttpMethod.Post,
    relativeUrl: string.Format(
        CultureInfo.InvariantCulture,
        IngestionClient.CreateFlightSubmissionUrlTemplate,
        IngestionClient.Version,
        IngestionClient.Tenant,

```

```

        appId,
        flightId),
        requestContent: null).Result;

// Update packages.
// Let's say we want to delete the existing package:
flightSubmission.flightPackages[0].fileStatus = "PendingDelete";

// Let's add a new package.
var packages = new List<dynamic>();
packages.Add(flightSubmission.flightPackages[0]);
packages.Add(
    new
    {
        fileStatus = "PendingUpload",
        fileName = "package.appx",
    });

flightSubmission.flightPackages =
JToken.FromObject(packages.ToArray());
var flightSubmissionId = flightSubmission.id.Value as string;

// Upload the zip archive with all new files to the SAS URL
returned with the submission.
var fileUploadUrl = flightSubmission.fileUploadUrl.Value as
string;
Console.WriteLine("FileUploadUrl: " + fileUploadUrl);
Console.WriteLine("Uploading file");
IngestionClient.UploadFileToBlob(@"..\..\files.zip",
fileUploadUrl).Wait();

// Update the submission.
Console.WriteLine("Updating the submission");
client.Invoke<dynamic>(
    HttpMethod.Put,
    relativeUrl: string.Format(
        CultureInfo.InvariantCulture,
        IngestionClient.GetFlightSubmissionUrlTemplate,
        IngestionClient.Version,
        IngestionClient.Tenant,
        appId,
        flightId,
        flightSubmissionId),
    requestContent: flightSubmission).Wait();

// Tell the system that we are done updating the submission.
Console.WriteLine("Committing the submission");
client.Invoke<dynamic>(
    HttpMethod.Post,
    relativeUrl: string.Format(
        CultureInfo.InvariantCulture,
        IngestionClient.CommitFlightSubmissionUrlTemplate,
        IngestionClient.Version,
        IngestionClient.Tenant,

```



```

        appId,
        flightId,
        flightSubmissionId),
        requestContent: null).Wait();

    // Periodically check the status until it changes from
    "CommitsStarted" to either
    // successful status or a failure.
    Console.WriteLine("Waiting for the submission commit processing
    to complete. This may take a couple of minutes.");
    string submissionStatus = null;
    do
    {
        Task.Delay(TimeSpan.FromSeconds(5)).Wait();
        dynamic statusResource = client.Invoke<dynamic>(
            HttpMethod.Get,
            relativeUrl: string.Format(
                CultureInfo.InvariantCulture,
                IngestionClient.FlightSubmissionStatusUrlTemplate,
                IngestionClient.Version,
                IngestionClient.Tenant,
                appId,
                flightId,
                flightSubmissionId),
            requestContent: null).Result;

        submissionStatus = statusResource.status.Value as string;
        Console.WriteLine("Current status: " + submissionStatus);
    }
    while ("CommitStarted".Equals(submissionStatus));

    if ("CommitFailed".Equals(submissionStatus))
    {
        Console.WriteLine("Submission has failed. Please check the
        Errors collection of the submissionResource response.");
        return;
    }
    else
    {
        Console.WriteLine("Submission commit success!");
    }
}
}
}

```

IngestionClient helper class

The `IngestionClient` class provides helper methods that are used by other methods in the sample app to perform the following tasks:

- [Obtain an Azure AD access token](#) that can be used to call methods in the Microsoft Store submission API. After you obtain a token, you have 60 minutes to use this token in calls to the Microsoft Store submission API before the token expires. After the token expires, you can generate a new token.
- Upload a ZIP archive containing new assets for an app or add-on submission to Azure Blob Storage. For more information about uploading a ZIP archive to Azure Blob Storage for app and add-on submissions, see the relevant instructions in [Create an app submission](#) and [Create an add-on submission](#).
- Process the HTTP requests for the Microsoft Store submission API.

C#

```
namespace DeveloperApiCSharpSample
{
    using System;
    using System.Collections.Generic;
    using System.IO;
    using System.Net.Http;
    using System.Net.Http.Headers;
    using System.Text;
    using System.Threading.Tasks;
    using Newtonsoft.Json;
    using Microsoft.WindowsAzure.Storage.Blob;

    /// <summary>
    /// This class is a proxy that abstracts the functionality of the API
    service
    /// </summary>
    public class IngestionClient : IDisposable
    {
        public static readonly string Version = "1.0";
        public static readonly string Tenant = "my";

        public static readonly string GetSubmissionUrlTemplate =
            "/v{0}/{1}/applications/{2}/submissions/{3}";
        public static readonly string CommitSubmissionUrlTemplate =
            "/v{0}/{1}/applications/{2}/submissions/{3}/commit";
        public static readonly string UpdateUrlTemplate =
            "/v{0}/{1}/applications/{2}/submissions/{3}/";
        public static readonly string ApplicationUrl =
            "/v{0}/{1}/applications";
        public static readonly string ApplicationUrlWithContinuation =
            "/v{0}/{1}/{2}";
        public static readonly string GetApplicationUrlTemplate =
            "/v{0}/{1}/applications/{2}";
        public static readonly string
            GetApplicationIapsWithContinuationUrlTemplate = "/v{0}/{1}/{2}";
        public static readonly string CreateSubmissionUrlTemplate =
            "/v{0}/{1}/applications/{2}/submissions";

        public static readonly string GetApplicationIapsUrlTemplate =
```

```

"/v{0}/{1}/applications/{2}/listinappproducts";
    public static readonly string CreateInAppUrlTemplate =
"/v{0}/{1}/inappproducts";
    public static readonly string GetInAppUrlTemplate =
"/v{0}/{1}/inappproducts/{2}";
    public static readonly string InAppSubmissionUrlTemplate =
"/v{0}/{1}/inappproducts/{2}/submissions/{3}";
    public static readonly string InAppSubmissionUrl =
"/v{0}/{1}/inappproducts/{2}/submissions";
    public static readonly string
InAppProductCommitSubmissionUrlTemplate =
"/v{0}/{1}/inappproducts/{2}/submissions/{3}/commit";

    public static readonly string GetApplicationFlightsUrlTemplate =
"/v{0}/{1}/applications/{2}/listflights";
    public static readonly string
GetApplicationFlightsWithContinuationUrlTemplate = "/v{0}/{1}/{2}";
    public static readonly string CreateNewFlightUrlTemplate =
"/v{0}/{1}/applications/{2}/flights";
    public static readonly string GetFlightUrlTemplate =
"/v{0}/{1}/applications/{2}/flights/{3}";
    public static readonly string CreateFlightSubmissionUrlTemplate =
"/v{0}/{1}/applications/{2}/flights/{3}/submissions";
    public static readonly string GetFlightSubmissionUrlTemplate =
"/v{0}/{1}/applications/{2}/flights/{3}/submissions/{4}";
    public static readonly string CommitFlightSubmissionUrlTemplate =
"/v{0}/{1}/applications/{2}/flights/{3}/submissions/{4}/commit";

    public static readonly string FlightSubmissionStatusUrlTemplate =
"/v{0}/{1}/applications/{2}/flights/{3}/submissions/{4}/status";
    public static readonly string ApplicationSubmissionStatusUrlTemplate
= "/v{0}/{1}/applications/{2}/submissions/{3}/status";
    public static readonly string InAppSubmissionStatusUrlTemplate =
"/v{0}/{1}/inappproducts/{2}/submissions/{3}/status";

    private HttpClient httpClient;
    private readonly string accessToken;

    /// <summary>
    /// Initializes a new instance of the <see cref="IngestionClient" />
class.
    /// </summary>
    /// <param name="accessToken">
    /// The acces token. This is JWT a token obtained from AAD allowing
the caller to invoke the API
    /// on behalf of a user
    /// </param>
    /// <param name="serviceUrl">The service URL.</param>
    public IngestionClient(string accessToken, string serviceUrl)
    {
        if (string.IsNullOrEmpty(accessToken))
        {
            throw new ArgumentNullException("accessToken");
        }
    }

```

```

        if (string.IsNullOrEmpty(serviceUrl))
        {
            throw new ArgumentNullException("serviceUrl");
        }

        this.accessToken = accessToken;
        this.httpClient = new HttpClient
        {
            BaseAddress = new Uri(serviceUrl)
        };
        this.DefaultHeaders = new Dictionary<string, string>();
    }

    /// <summary>
    /// Gets the default headers.
    /// </summary>
    public Dictionary<string, string> DefaultHeaders { get; private set; }
}

    /// <summary>
    /// Performs application-defined tasks associated with freeing,
    releasing, or resetting
    /// unmanaged resources.
    /// </summary>
    public void Dispose()
    {
        if (this.httpClient != null)
        {
            this.httpClient.Dispose();
            this.httpClient = null;
            GC.SuppressFinalize(this);
        }
    }

    /// <summary>
    /// Gets the authorization token for the provided client id, client
    secret, and the scope.
    /// This token is usually valid for 1 hour, so if your submission
    takes longer than that to complete,
    /// make sure to get a new one periodically.
    /// </summary>
    /// <param name="tokenEndpoint">Token endpoint to which the request
    is to be made. Specific to your
    /// AAD app. Example: https://login.windows.net/d454d300-128e-2d81-
    334a-27d9b2baf002/oauth2/token </param>
    /// <param name="clientId">Client Id of your AAD app. Example"
    ba3c223b-03ab-4a44-aa32-38aa10c27e32</param>
    /// <param name="clientSecret">Client secret of your AAD app</param>
    /// <param name="scope">Scope. If not provided, default one is used
    for the production API endpoint.</param>
    /// <returns>Authorization token. Prepend it with "Bearer: " and pass
    it in the request header as the
    /// value for "Authorization: " header.</returns>
    public static async Task<string> GetClientCredentialAccessToken(

```

```

        string tokenEndpoint,
        string clientId,
        string clientSecret,
        string scope = null)
    {
        if (scope == null)
        {
            scope = "https://manage.devcenter.microsoft.com";
        }

        dynamic result;
        using (HttpClient client = new HttpClient())
        {
            string tokenUrl = tokenEndpoint;
            using (
                HttpRequestMessage request = new HttpRequestMessage(
                    HttpMethod.Post,
                    tokenUrl))
            {
                string strContent =
                    string.Format(
                        "grant_type=client_credentials&client_id={0}&client_secret={1}&resource={2}",
                        clientId,
                        clientSecret,
                        scope);

                request.Content = new StringContent(strContent,
Encoding.UTF8,
                    "application/x-www-form-urlencoded");

                using (HttpResponseMessage response = await
client.SendAsync(request))
                {
                    string responseContent = await
response.Content.ReadAsStringAsync();
                    result =
JsonConvert.DeserializeObject(responseContent);
                }
            }

            return result.access_token;
        }

        /// <summary>
        /// Uploads a file to blob using a SAS url
        /// </summary>
        /// <param name="fileName">Path to your zip file</param>
        /// <param name="sasUrl">The SAS url which was returned to you when
you cloned the submission
        /// in FileUploadUrl</param>
        /// <returns>A task which will complete when the file finishes
uploading</returns>
        public static async Task UploadFileToBlob(string fileName, string

```

```

sasUrl)
{
    using (Stream stream = new FileStream(fileName, FileMode.Open))
    {
        var blockBob = new CloudBlockBlob(new Uri(sasUrl));
        await blockBob.UploadFromStreamAsync(stream);
    }
}

/// <summary>
/// Invokes the specified HTTP method.
/// </summary>
/// <typeparam name="T"></typeparam>
/// <param name="httpMethod">The HTTP method.</param>
/// <param name="relativeUrl">The relative URL.</param>
/// <param name="requestContent">Content of the request.</param>
/// <returns>instance of the type T</returns>
/// <exception cref="ServiceException"></exception>
public async Task<T> Invoke<T>(HttpMethod httpMethod,
    string relativeUrl,
    object requestContent)
{
    using (var request = new HttpRequestMessage(httpMethod,
relativeUrl))
    {
        this.SetRequest(request, requestContent);

        using (HttpResponseMessage response = await
this.httpClient.SendAsync(request))
        {
            T result;
            if (this.TryHandleResponse(response, out result))
            {
                return result;
            }

            if (response.IsSuccessStatusCode)
            {
                var resource = JsonConvert.DeserializeObject<T>
(await response.Content.ReadAsStringAsync());
                return resource;
            }

            throw new
Exception(response.Content.ReadAsStringAsync().Result);
        }
    }
}

/// <summary>
/// Sets the request.
/// </summary>
/// <param name="request">The request.</param>
/// <param name="requestContent">Content of the request.</param>
protected virtual void SetRequest(HttpRequestMessage request, object

```

```

requestContent)
{

request.Headers.Add(Constants.RequestHeaders.CorrelationIdHeader,
Guid.NewGuid().ToString());
        request.Headers.Add(Constants.RequestHeaders.MSRequestIdHeader,
Guid.NewGuid().ToString());

        request.Headers.Authorization = new
AuthenticationHeaderValue("Bearer", this.accessToken);

        foreach (var header in this.DefaultHeaders)
        {
            request.Headers.Add(header.Key, header.Value);
        }

        if (requestContent != null)
        {
            request.Content = new
StringContent(JsonConvert.SerializeObject(requestContent),
                Encoding.UTF8,
                Constants.HttpMimeTypes.JsonContentType);
        }
    }

    /// <summary>
    /// Tries the handle response.
    /// </summary>
    /// <typeparam name="T"></typeparam>
    /// <param name="response">The response.</param>
    /// <param name="result">The result.</param>
    /// <returns>true if the response was handled</returns>
    protected virtual bool TryHandleResponse<T>(HttpResponseMessage
response, out T result)
    {
        result = default(T);
        return false;
    }

    private static class Constants
    {
        public static class RequestHeaders
        {
            /// <summary>
            /// Corresponds to TraceCorrelationId in SLL. This is a GUID
that is newly generated
            /// by FD for every request coming from the client.
            /// </summary>
            public const string CorrelationIdHeader = "MS-
CorrelationId";

            /// <summary>
            /// Corresponds to RequestCorrelationId in SLL. This is a

```